

Universidad de La Habana
Facultad de Matemática y Computación

Proyecto de Simulación

Estrategias evolutivas bajo presión de depredación

Estudiante: David Sánchez Iglesias
Grupo: C411
Asignatura: Simulación
Año académico: 4.º año, Curso 2025–2026

Repositorio GitHub:

https://github.com/Davidbestion/Simulation_cowardice_vs_altruism

Índice

1. Introducción	2
2. Modelo	2
2.1. Entidades y estado del sistema	2
2.2. Dinámica por generación	3
2.3. Estrategias génicas consideradas	3
2.4. Supuestos y alcance	4
3. Implementación	4
3.1. Estructura general del código	4
3.2. Mundo y árboles	5
3.3. Criaturas y genoma	5
3.4. Genes y comportamientos	5
3.5. Ciclo de simulación (por generación)	7
3.6. Asignación de herbívoros a árboles	7
3.7. Depredadores y detección	8
3.8. Generación de genomas y experimento	8
3.9. Reproducción y manejo del genoma	8
3.10. Inicialización, semilla y reproducibilidad	8
3.11. Reportes	9
3.12. Notas de extensibilidad	9
4. Resultados experimentales	9
5. Discusión global	10
5.1. Inestabilidad del altruismo puro y condiciones para el altruismo selectivo	10
5.2. Camuflaje y consecuencias diferenciales en herbívoros y depredadores	10
5.3. Sociabilidad, cobardía solitaria y la trampa de la depredación grupal	10
5.4. Robustez del tamaño poblacional	11
6. Conclusiones	11

1. Introducción

Este trabajo estudia, mediante simulación computacional basada en agentes, la competencia y coevolución de distintas estrategias de conducta en un ecosistema con depredación activa y recursos limitados. El punto de partida es la pregunta clásica de la biología evolutiva: bajo qué condiciones puede mantenerse el altruismo cuando comparte población con individuos que se benefician sin asumir costes. Sin embargo, el estudio se extiende considerablemente más allá: se incorporan mecanismos de camuflaje en herbívoros y depredadores, estrategias de agrupamiento social, señales de reconocimiento y una población de depredadores con genoma propio que coevoluciona con la presa.

El modelo implementado simula dos poblaciones independientes —herbívoros y depredadores— cuyo comportamiento emerge de su genoma. Los genes de los herbívoros pueden definir estrategias de reacción ante el peligro (cobardía, altruismo puro, altruismo selectivo con barba verde), rasgos de camuflaje que dificultan la detección, o preferencias de agrupamiento (social o solitario). Los genes de los depredadores determinan cómo eligen los árboles donde cazar —estrategias de emboscada sobre grupos, caza cautelosa sobre solitarios— y si poseen camuflaje que reduce la probabilidad de ser detectados por la presa. Ambas poblaciones se reproducen y evolucionan de forma independiente en cada generación, dando lugar a dinámicas de coevolución presa–depredador.

El objetivo del informe es triple: (i) describir el modelo y la arquitectura del simulador; (ii) presentar el diseño de los nueve experimentos realizados, de complejidad creciente; y (iii) discutir los patrones evolutivos observados, distinguiendo resultados robustos de los dependientes de la estocasticidad. Los resultados cuantitativos completos, las gráficas de evolución génica y las tablas de fijación y extinción por experimento se encuentran en el cuaderno de experimentación `experimentacion_core.ipynb`.

2. Modelo

2.1. Entidades y estado del sistema

El sistema consta de tres tipos de entidades: **herbívoros**, **depredadores** y **árboles**.

Cada herbívoro posee un *genoma* (lista de genes) del que se derivan:

- **physical_traits**: rasgos físicos como **green_beard** (señal de reconocimiento), **hidden_from_predator** (invisibilidad ante la elección de árbol del depredador) y **hunt_evasion** (probabilidad adicional de evadir la caza una vez iniciada).
- **behavioral_traits**: preferencias de asignación a árboles (buscar árboles con más o menos ocupantes según la estrategia social).
- Comportamiento ante depredador: resolución del enfrentamiento cuando hay un depredador en el árbol.

De forma análoga, cada depredador posee su propio genoma, que puede definir:

- Estrategia de elección de árbol (**predator_choose_tree**): por ejemplo, elegir el árbol más poblado (emboscada) o el menos poblado (caza cautelosa).
- Rasgos físicos como **camouflage_factor**, que reduce la probabilidad de ser detectado por los herbívoros.

El entorno contiene **num_trees** árboles. La capacidad máxima de alimentación de cada árbol se sortea al inicio de cada generación entre **tree_capacity_min** y **tree_capacity_max**. La mortalidad por generación resulta de dos mecanismos: *hambre* (exceso de población respecto a la capacidad total de los árboles) y *depredación*.

2.2. Dinámica por generación

La simulación opera en generaciones no solapadas. En cada generación se ejecutan, en orden:

1. **Asignación de herbívoros a árboles:** cada herbívoro elige un slot según sus preferencias génicas (social: prefiere árboles llenos; solitario: prefiere árboles vacíos; sin preferencia: distribución aleatoria). Si la población excede la capacidad total, el excedente muere por hambre.
2. **Asignación de depredadores a árboles:** cada depredador elige un árbol para cazar según su estrategia génica (`predator_choose_tree`). Los herbívoros con el rasgo `hidden_from_predator` no son visibles para esta elección.
3. **Detección y enfrentamiento:** en cada árbol con depredador, cada herbívoro intenta detectarlo de forma independiente con probabilidad $p = \text{base_detection_prob} \times (1 - \text{camouflage_factor}_{\text{pred}})$. Si al menos uno detecta, se elige aleatoriamente un *notificador* entre ellos y se ejecuta su `predator_behavior`. Si nadie detecta, el depredador caza libremente hasta su capacidad (`predator_hunt_capacity`). Los herbívoros con `hunt_evasion` tienen una probabilidad adicional de escapar durante la caza. El efecto de “muchos ojos” surge de forma natural: $P(\text{detección grupal}) = 1 - (1 - p)^N$ para un grupo de N herbívoros.
4. **Reproducción de herbívoros:** los supervivientes generan entre `offspring_min` y `offspring_max` descendientes con el mismo genoma; los progenitores no pasan a la siguiente generación.
5. **Reproducción de depredadores:** los depredadores que cazaron al menos una presa generan entre `predator_offspring_min` y `predator_offspring_max` descendientes. Los que no cazaron mueren de inanición.
6. **Registro:** se computan las frecuencias génicas de herbívoros y depredadores por separado y se almacena un `GenerationReport` con todas las métricas de la generación.

2.3. Estrategias génicas consideradas

Los genes se dividen en dos grupos según la especie que los porta.

Genes de herbívoros:

- **CowardGene:** el notificador huye; los demás ocupantes del árbol mueren.
- **AltruistGene:** avisa a todos; su propia supervivencia ocurre con probabilidad `altruist_escape_prob`.
- **SelectiveAltruistGene:** altruismo selectivo solo hacia portadores de `green_beard`.
- **GreenBeardGene:** aporta la señal física `green_beard` sin comportamiento altruista.
- **GreenBeardAltruistGene:** combina señal `green_beard` y altruismo selectivo.
- **CamouflageHerbivoreGene:** hace al herbívoro invisible para la elección de árbol del depredador (`hidden_from_predator`) y le confiere evasión durante la caza (`hunt_evasion = 0.4`).
- **SocialGene:** prefiere árboles con más ocupantes (agrupamiento social).
- **SolitaryGene:** prefiere árboles con menos ocupantes (dispersión solitaria).
- **SocialGreenBeardAltruistGene:** gen compuesto; combina agrupamiento social, señal de barba verde y altruismo selectivo.
- **SolitaryCowardGene:** gen compuesto; combina dispersión solitaria y conducta cobarde.

Genes de depredadores:

- **CamouflagePredatorGene**: reduce la probabilidad de ser detectado por los herbívoros (`camouflage_factor=0.6`).
- **AmbushGene**: elige el árbol con más herbívoros visibles (emboscada en grupos).
- **CautiousPredatorGene**: elige el árbol con menos herbívoros visibles (caza solitaria).
- **GreedyHunterGene**: aumenta el número de presas capturadas por visita.
- **AmbushCamoGene**: gen compuesto; combina camuflaje extremo (`camouflage_factor=0.9`) con estrategia de emboscada en el árbol más poblado.

Cuando una criatura posee múltiples genes con comportamiento frente a depredador, se aplica una regla de prioridad (campo `PRIORITY`), ejecutándose el gen de mayor prioridad que produzca una resolución válida.

2.4. Supuestos y alcance

El modelo asume herencia directa del genoma sin mutación, con apareamiento implícito y generaciones no solapadas (los progenitores no conviven con sus descendientes). La asignación a árboles y la detección del depredador son procesos estocásticos independientes entre generaciones.

Bajo estos supuestos, el simulador no pretende reproducir ninguna especie concreta, sino ofrecer un entorno controlado para estudiar cómo distintos rasgos —cooperación, camuflaje, agrupamiento y especialización depredadora— afectan a la estabilidad evolutiva y a la dinámica coevolutiva presa-depredador. El alcance experimental abarca desde los mecanismos clásicos de la teoría del altruismo hasta fenómenos más complejos como el suicidio evolutivo del depredador camuflado o la paradoja del agrupamiento social frente a depredadores especializados en grupos.

3. Implementación

3.1. Estructura general del código

El código principal de la simulación se encuentra en `src/simulation/core` y está organizado por responsabilidad:

- `simulation.py`: configuración (`SimulationConfig`), contrato de experimento (`Experiment`), registro diario (`DayReport/GenerationReport`) y `TreeInteractionLog`.
- `world.py`: clases `Tree` y `World`.
- `creature.py`: clases `Creature`, `Herbivore` y `Predator`.
- `genes.py`: la API base `Gene` y las implementaciones de los 10 genes de herbívoros y 5 genes de depredadores descritos en la sección 3.4.
- `simulator.py`: ciclo diurno y recogida de `DayReport`.

En adición, la carpeta `src/simulation/instances` contiene los experimentos y sus generadores de genomas, y `src/simulation/analysis` las funciones de visualización.

3.2. Mundo y árboles

El entorno se modela con las clases `Tree` y `World` (archivo `world.py`). Cada `Tree` tiene dos atributos:

- `tree_id`: identificador entero único dentro del mundo.
- `capacity`: máximo de herbívoros que pueden alimentarse en ese árbol por día.

La clase `World` agrupa la colección de árboles y expone:

- `num_trees` (propiedad): número total de árboles.
- `total_capacity()` (método): suma de las capacidades de todos los árboles.
- `World.create(num_trees, capacity_min, capacity_max, rng)`: método estático que construye un nuevo mundo sorteando la capacidad de cada árbol de forma independiente.

El mundo se crea **una sola vez** al inicio de la simulación (antes del primer día) y sus árboles permanecen fijos durante toda la ejecución.

3.3. Criaturas y genoma

Hay dos subclases concretas de `Creature` (archivo `creature.py`): `Herbivore` y `Predator`. Ambas se construyen a partir de un `genome` (lista de instancias de `Gene`) y comparten la misma interfaz de rasgos:

- `physical_traits`: diccionario acumulado llamando `gene.apply_physical_traits(creature)` sobre cada gen del genoma. Es el único dict de rasgos; no existe un dict separado de rasgos de comportamiento.
- `has_trait(trait)`: devuelve `True` si el rasgo existe y tiene valor *truthy*.
- `get_trait_value(trait, default)`: devuelve el valor numérico del rasgo (útil para rasgos como `hunt_evasion` o `camouflage_factor`).
- `has_gene(gene_type)`: comprueba si hay al menos una instancia del tipo de gen en el genoma.

`Creature.reproduce(rng)` crea los descendientes al final del día: el número se sortea en `[min_offspring, max_offspring]` usando `rng`; los descendientes heredan el genoma completo sin modificación (no hay mutación ni recombinación).

3.4. Genes y comportamientos

El contrato de un gen está definido por la clase abstracta `Gene` (archivo `genes.py`). Los atributos de clase y métodos sobrescribibles son:

- `name`: nombre del gen (clave en los contadores de frecuencia génica).
- `PRIORITY`: entero; mayor prioridad se evalúa primero cuando varios genes compiten.
- `apply_physical_traits(creature) → dict`: rasgos físicos que aporta el gen (`{}` por defecto, sin efecto).
- `choose_tree(creature, available_trees, all_assignments, cfg, rng) → int|None`: preferencia de árbol del herbívoro; `None` para deferir al siguiente gen.

- `predator_behavior(notifier, assigned, cfg, rng) → (fled, directly_eaten)|None`: respuesta del herbívoro al detectar un depredador; ambas listas contienen instancias de `Herbivore`; `None` para deferir.
- `predator_choose_tree(predator, trees, visible_herbivores, cfg, rng) → int|None`: elección de árbol del depredador; `None` para deferir.
- `modify_hunt_capacity(predator, current_capacity, cfg) → int`: ajuste del número máximo de presas que captura el depredador por visita.

La API de `Gene` se ha extendido respecto al diseño original para soportar tanto herbívoros como depredadores:

- `predator_choose_tree(predator, trees, visible_herbivores, cfg, rng)`: solo para depredadores; devuelve el `tree_id` elegido para cazar.
- `choose_tree(creature, available_trees, all_assignments, cfg, rng)`: para herbívoros con preferencias de asignación (social, solitario).

Genes de herbívoros implementados:

- `CowardGene` (`PRIORITY=10`): el notificante huye; los demás mueren.
- `AltruistGene` (`PRIORITY=20`): avisa a todos; escapa con probabilidad `altruist_escape_prob`.
- `SelectiveAltruistGene` (`PRIORITY=25`): altruismo selectivo hacia portadores de `green_beard`.
- `GreenBeardGene` (`PRIORITY=5`): aporta el rasgo físico `green_beard`.
- `GreenBeardAltruistGene` (`PRIORITY=30`): combina `green_beard` + altruismo selectivo.
- `CamouflageHerbivoreGene` (`PRIORITY=10`): `hidden_from_predator=True`, `hunt_evasion=0.4`; comportamiento cobarde.
- `SocialGene`: `choose_tree` elige el árbol con más ocupantes.
- `SolitaryGene`: `choose_tree` elige el árbol con menos ocupantes.
- `SocialGreenBeardAltruistGene` (`PRIORITY=35`): gen compuesto; social + barba verde + altruismo selectivo.
- `SolitaryCowardGene` (`PRIORITY=35`): gen compuesto; solitario + cobarde.

Genes de depredadores implementados:

- `CamouflagePredatorGene` (`PRIORITY=10`): `camouflage_factor=0.6`.
- `AmbushGene` (`PRIORITY=15`): `predator_choose_tree` elige el árbol más poblado.
- `CautiousPredatorGene` (`PRIORITY=15`): `predator_choose_tree` elige el árbol menos poblado.
- `GreedyHunterGene` (`PRIORITY=10`): aumenta `predator_hunt_capacity`.
- `AmbushCamoGene` (`PRIORITY=20`): gen compuesto; `camouflage_factor=0.9` + emboscada en grupo más poblado.

`Creature.handle_predator` ordena los genes que implementan `predator_behavior` por `PRIORITY` descendente y ejecuta en ese orden hasta obtener una resolución válida.

3.5. Ciclo de simulación (por generación)

La clase `Simulator` (archivo `simulator.py`) implementa el ciclo principal en `run()`:

1. Inicialización del RNG con `cfg.seed` (si no es `None`) para reproducibilidad.
2. Creación del `World` con `World.create()` (única vez en toda la simulación).
3. Llamadas a `_build_population()` usando `experiment.herbivore_specs` y `experiment.predator_specs` para construir las poblaciones iniciales.
4. Bucle de generaciones `for gen in 1..cfg.num_generations:`
 - a) Crear el `World` y los objetos `Tree` para esta generación; la capacidad por árbol se sortea entre `tree_capacity_min` y `tree_capacity_max`.
 - b) Asignar herbívoros a árboles según sus preferencias génicas (`choose_tree`). Si la población excede la capacidad total, el excedente muere por hambre (`starved`).
 - c) Asignar depredadores a árboles según `predator_choose_tree`; los herbívoros con `hidden_from_predator` no son visibles en esta fase.
 - d) Para cada árbol con al menos un depredador, resolver el enfrentamiento (detección independiente por herbívoro, notificación, caza libre si no hay detección) según se describe en la sección 3.7.
 - e) Los herbívoros supervivientes se reproducen con `Creature.reproduce()`; los progenitores no pasan a la siguiente generación.
 - f) Los depredadores que cazaron al menos una presa se reproducen; los que no cazaron mueren de inanición.
 - g) Se computan las frecuencias génicas de herbívoros y depredadores y se almacena un `GenerationReport`. Si alguna población queda vacía, el bucle puede terminar anticipadamente.

3.6. Asignación de herbívoros a árboles

La asignación sigue un esquema greedy secuencial con orden aleatorio:

1. Se baraja la lista de herbívoros (`rng.shuffle`) para eliminar cualquier sesgo sistemático por orden de creación.
2. Cada herbívoro, por turno, consulta su gen de elección (`choose_tree`) recibiendo únicamente los árboles que aún tienen capacidad libre. Si el gen devuelve un árbol ya lleno (caso de seguridad), se redirige a uno aleatorio entre los disponibles.
3. El herbívoro ocupa un slot del árbol elegido, reduciendo en 1 su capacidad restante.
4. Si en el momento de su turno no queda ningún slot libre en ningún árbol, el herbívoro muere de hambre y no participa en ninguna fase posterior.

Este mecanismo garantiza que la capacidad por árbol se respete y que la hambruna surja de forma natural cuando la población supera la suma de capacidades, sin necesidad de calcular el excedente explícitamente.

3.7. Depredadores y detección

En el modelo extendido, los depredadores son agentes activos con genoma propio. En cada generación, cada depredador elige un árbol según su `predator_choose_tree`: los herbívoros con el rasgo `hidden_from_predator` no son visibles para esta elección y no influyen en ella.

Una vez asignados los depredadores a árboles, se resuelve el enfrentamiento árbol a árbol:

1. **Detección.** Se calcula la probabilidad efectiva de detección como $p_{\text{eff}} = p_{\text{base}} \cdot (1 - f_{\text{camo_max}})$, donde $f_{\text{camo_max}}$ es el `camouflage_factor` del depredador *más camuflado* presente en el árbol. Los herbívoros se procesan en orden aleatorio; el **primero** que supera el umbral p_{eff} se convierte en el *detector*.
2. **Alarma.** El detector ejecuta `handle_predator()`, que devuelve (`fled`, `directly_eaten`): herbívoros que huyen antes de la caza y herbívoros devorados directamente (p. ej. el altruista que se sacrifica). Los demás permanecen expuestos a la fase de caza.
3. **Caza.** Los depredadores atacan en orden aleatorio sobre los herbívoros que no huyeron ni fueron devorados directamente. Cada depredador captura hasta `predator_hunt_capacity` presas (modificable por `modify_hunt_capacity`); cada presa puede escapar individualmente si posee `hunt_evasion` > 0. Las presas capturadas se retiran del pool antes del siguiente depredador.
4. Solo los depredadores que cazaron al menos una presa sobreviven y se reproducen.

El `GenerationReport` registra las frecuencias génicas de herbívoros y depredadores por separado (`gene_frequencies` y `predator_gene_frequencies`).

3.8. Generación de genomas y experimento

Cada `Experiment` define dos propiedades de especificación de poblaciones:

- `herbivore_specs`: lista de tuplas (`genome_factory`, `fraction`) para los herbívoros. `genome_factory` es un callable sin argumentos que devuelve una lista de genes; `fraction` es la proporción de la población inicial que tendrá ese genoma.
- `predator_specs`: misma estructura para los depredadores (lista vacía por defecto si el experimento no tiene depredadores explícitos).

`Simulator._build_population()` convierte las fracciones en conteos enteros (con corrección de redondeo en el último subtipo) y construye las instancias de `Herbivore` o `Predator`. El alias `population_specs` sigue funcionando por compatibilidad con código anterior.

3.9. Reproducción y manejo del genoma

`Creature.reproduce(rng)` crea descendientes con `list(self.genome)`: se hace una copia superficial de la lista, lo que es suficiente mientras los genes sean objetos inmutables compartidos. Los progenitores no se incluyen en la lista devuelta; la nueva generación está compuesta exclusivamente por los descendientes.

3.10. Inicialización, semilla y reproducibilidad

`SimulationConfig` agrupa todos los parámetros de la simulación. Los más relevantes son: `num_trees`, `tree_capacity_min/max`, `num_generations`, `initial_population`, `initial_predators`, `offspring_min/max`, `predator_offspring_min/max`, `base_detection_prob`, `altruist_escape_prob`, `predator_hunt_capacity` y `seed`.

`Simulator.run()` instancia un `random.Random(cfg.seed)` propio (RNG aislado, no el global del módulo `random`) y lo pasa explícitamente a cada fase. Esto garantiza que dos ejecuciones con la misma semilla produzcan resultados idénticos con independencia del estado global del intérprete.

3.11. Reportes

Por cada día `Simulator` construye un `DayReport` (clase definida en `simulation.py`; el alias `GenerationReport` se mantiene por compatibilidad). Sus campos principales son:

- `generation / day`: número de día (base 1).
- `population_start`, `starved`, `eaten`, `survived`, `population_end`: estadísticas de los herbívoros.
- `gene_frequencies`: mapa `nombre_gen → frecuencia` para herbívoros.
- `predator_count_start`, `predator_count_end`: tamaño de la población de depredadores al inicio y al final del día.
- `predator_gene_frequencies`: frecuencias génicas de los depredadores.
- `tree_interactions`: lista de `TreeInteractionLog`, uno por árbol con herbívoros, con el detalle de detección, alarma y caza de ese árbol ese día.

3.12. Notas de extensibilidad

- Añadir un nuevo comportamiento requiere crear una subclase de `Gene` e implementar los métodos relevantes (`predator_behavior`, `choose_tree`, `predator_choose_tree`, `modify_hunt_capacity`); el resto del sistema lo descubrirá automáticamente por reflexión (`hasattr`).
- Si se introdujera mutación, `Creature.reproduce()` debería clonar en profundidad los genes mutables para evitar aliasing entre generaciones.
- El campo `predator_probability` de `SimulationConfig` se conserva por compatibilidad con configuraciones antiguas pero no es usado por el simulador extendido; la probabilidad de encuentro con depredador la controla `base_detection_prob` junto con las estrategias de elección de árbol de los depredadores.

4. Resultados experimentales

Los experimentos se diseñaron con una lógica de complejidad creciente: partiendo de un escenario homogéneo de referencia, se introdujo progresivamente la competencia entre estrategias y se añadieron mecanismos de camuflaje, reconocimiento y cooperación selectiva. Los parámetros del entorno son comunes a todos los experimentos salvo indicación contraria (Tabla 1).

Cuadro 1: Parámetros fijos del entorno de simulación.

Parámetro	Valor
Número de árboles	25
Población inicial	80 individuos
Probabilidad de depredador por árbol	0,30
Crías por superviviente por generación	1–2
Semilla aleatoria (<code>seed</code>)	42

El diseño experimental abarca nueve experimentos de complejidad creciente. Los tres primeros estudian la dinámica de cobardía pura, altruismo puro y altruismo selectivo con señal de barba verde, respectivamente, variando la probabilidad de escape del altruista y la proporción inicial de portadores. Los tres siguientes introducen el camuflaje como variable experimental: en herbívoros (Exp. 5), en depredadores (Exp. 6) y en ambas especies de forma simultánea (Exp. 7). Los dos últimos experimentos exploran combinaciones más complejas de estrategias sociales y especialización de nichos depredadores (Exp. 8 y 9).

El detalle completo del diseño, los parámetros por combinación, los resultados numéricos y las conclusiones de cada experimento se encuentran en el cuaderno de experimentación: `experimentacion_core.ipynb`, incluido en el repositorio del proyecto. Dicho cuaderno documenta 30 corridas independientes por combinación, las gráficas de evolución genética y coevolución poblacional, y las tablas de fijación y extinción por experimento.

A continuación se presenta una discusión global de los patrones observados.

5. Discusión global

5.1. Inestabilidad del altruismo puro y condiciones para el altruismo selectivo

Los experimentos 1–4 confirman que el altruismo puro es evolutivamente inestable: los cobardes explotan el aviso de peligro sin asumir coste, desplazando sistemáticamente a los altruistas independientemente de su probabilidad de escape (Exp. 2–4). El altruismo selectivo con señal de barba verde supera este problema cuando señal y comportamiento permanecen acoplados: la señal garantiza que el coste se dirija exclusivamente hacia otros cooperadores, generando una ventaja neta sobre los cobardes (Exp. 3). Sin embargo, este mecanismo muestra dependencia de masa crítica: cuando los altruistas parten de una minoría del 10 %, la viabilidad depende del coste individual; con p_{escape} baja, la escasez de encuentros entre portadores produce extinción. El desacoplamiento total de señal y conducta (Exp. 4) colapsa ambas estrategias altruistas antes de la generación 50, en consonancia con los modelos teóricos de barba verde [3].

5.2. Camuflaje y consecuencias diferenciales en herbívoros y depredadores

Introducir el camuflaje produce resultados asimétricos según la especie. En herbívoros (Exp. 5), el camuflaje reduce la probabilidad de ser detectado y, combinado con estrategias sociales (Exp. 7–8), refuerza la supervivencia grupal. En depredadores (Exp. 6–9), el camuflaje produce el efecto opuesto: la mayor eficiencia depredadora depleta la presa más rápido de lo que el ecosistema puede recuperarla, induciendo el colapso de la propia población depredadora —un caso de suicidio evolutivo o “paradoja de la eficiencia”—. La dinámica de Lotka–Volterra subyacente amplifica esta inestabilidad: en poblaciones pequeñas, las fluctuaciones estocásticas llevan al depredador a la extinción incluso cuando su adaptación individual es óptima.

5.3. Sociabilidad, cobardía solitaria y la trampa de la depredación grupal

Los experimentos 8 y 9 revelan una tensión entre la ventaja colectiva de la sociabilidad y el riesgo de convertirse en objetivo preferido. El gen `SocialGreenBeardAltruistGene` (Exp. 8) aprovecha el efecto de “muchos ojos”: en un grupo de N herbívoros con probabilidad individual de detección p , la probabilidad de que al menos uno detecte al depredador es $1 - (1 - p)^N$, proporcionando una ventaja de vigilancia colectiva creciente con el tamaño del grupo. Sin embargo, el Exp. 9 revela que esta ventaja puede invertirse: los depredadores con `AmbushCamoGene` eligen preferentemente los árboles con más herbívoros, anulando parcialmente el beneficio de la agrupación. Ante esta presión selectiva, la estrategia solitaria–cobarde (`SolitaryCowardGene`)

—que evita los árboles ya ocupados— reduce la exposición al depredador grupal, favoreciendo su dominancia a largo plazo.

5.4. Robustez del tamaño poblacional

Una observación transversal a todos los experimentos es que la composición genética determina la *estructura* de la población (qué estrategia domina), pero no su *tamaño*. En todos los escenarios la abundancia oscila en rangos similares, determinada fundamentalmente por los parámetros ecológicos del entorno: capacidad de los árboles, tasa reproductiva y probabilidad de depredación por árbol, no por la identidad del gen dominante.

6. Conclusiones

1. El altruismo puro no es evolutivamente estable: los cobardes explotan el aviso colectivo sin asumir coste, desplazando sistemáticamente al altruista, independientemente de su probabilidad de escape (Exp. 1–4).
2. El altruismo selectivo con barba verde puede fijarse cuando señal y comportamiento están acoplados, pero exhibe dependencia de masa crítica: una proporción inicial reducida de altruistas puede resultar en extinción si el coste individual es alto (Exp. 3).
3. El desacoplamiento entre señal y conducta altruista colapsa ambas estrategias en menos de 50 generaciones, confirmando que la estabilidad del mecanismo de barba verde requiere fiabilidad de la señal [3] (Exp. 4).
4. El camuflaje en depredadores produce suicidio evolutivo: la mayor eficiencia depredadora depleta la presa y conduce a la extinción del propio depredador (Exp. 6–9).
5. La ventaja colectiva de “muchos ojos” puede invertirse cuando depredadores especializados dirigen su atención a los grupos más numerosos, favoreciendo estrategias solitarias–cobardes (Exp. 9).
6. El tamaño de la población está determinado por los parámetros ecológicos del entorno, no por la composición genética (observación transversal a los nueve experimentos).

Los resultados obtenidos sugieren que la evolución del altruismo requiere mecanismos de reconocimiento difíciles de falsificar —como el parentesco genético o la reciprocidad en interacciones repetidas—, ninguno de los cuales ha sido modelado en el presente trabajo y constituye una línea natural de extensión futura. La evolución de estrategias depredadoras merece profundización adicional: la eficiencia depredadora excesiva resulta maladaptativa a escala de población, un resultado con implicaciones para modelos de gestión de ecosistemas.

Referencias

- [1] W.D. Hamilton, *The genetical evolution of social behaviour I & II*, Journal of Theoretical Biology, 7(1):1–52, 1964.
- [2] R. Dawkins, *The Selfish Gene*, Oxford University Press, Oxford, 1976.
- [3] A. Zahavi, *Mate selection—a selection for a handicap*, Journal of Theoretical Biology, 53(1):205–214, 1975.